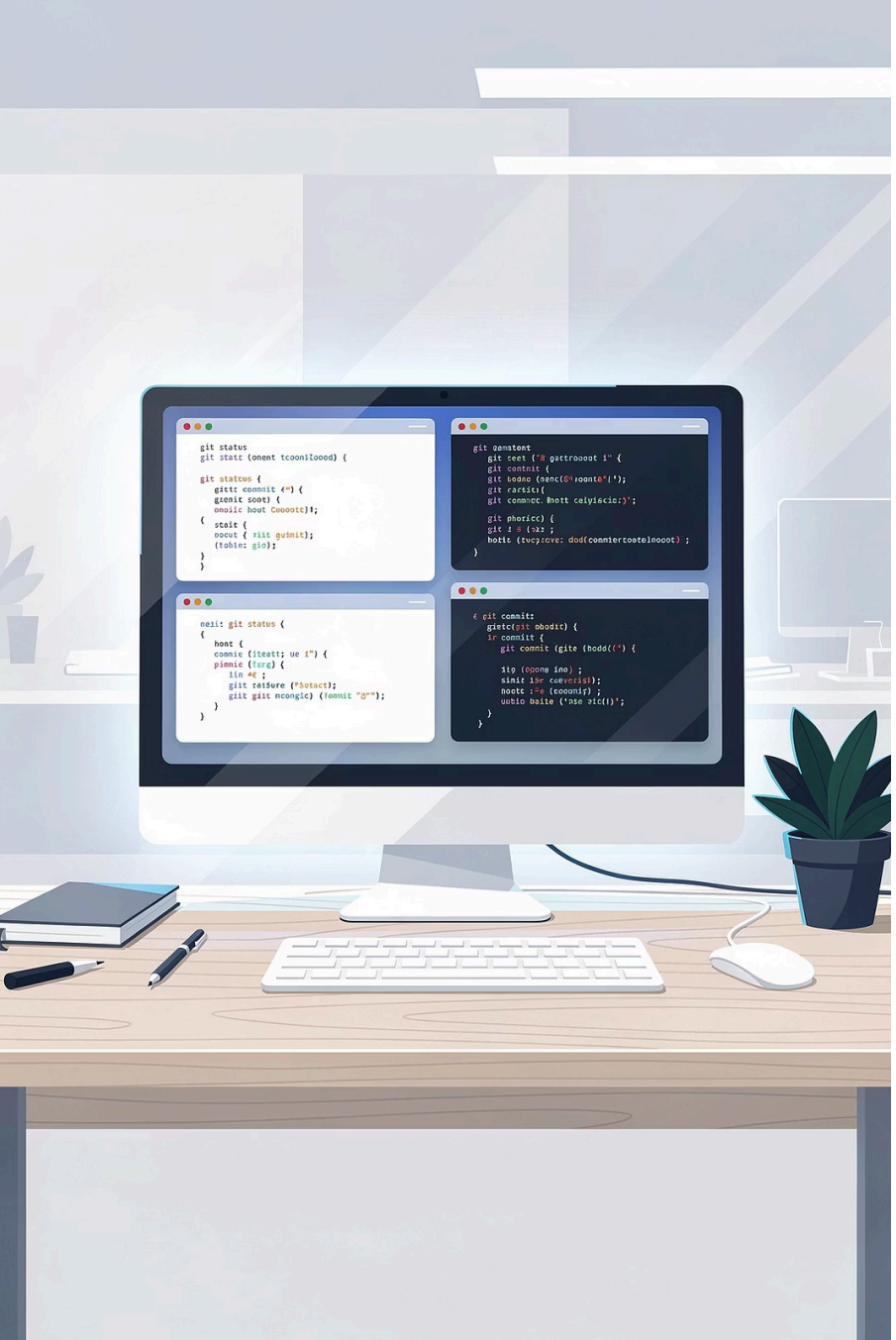


פקודות Git נוספות

מדריך מקיף לפקודות Git מתקדמות – מ-Stashing ועד פתרון קונפליקטים



שמירה זמנית של שינויים
Stashing

לפעמים אנחנו באמצע עבודה ורוצים לעבור לברנצ' אחר או לבצע פעולה אחרת, בלי לאבד את השינויים שעשינו. כאן נכנס ה-Stash.

1

שמירת שינויים

```
git stash
```

שומר את השינויים הזמניים בצד ומנקה את ה-Working Directory

2

שחזור שינויים

```
git stash pop
```

מחזיר את השינויים ששמרנו מה-stash ומסיר אותם מה-stash

החזרת קובץ מ-Staging ל-Working Directory

כאשר הוספנו קובץ ל-Staging בטעות ורוצים להחזיר אותו:

```
git rm --cached index.html
```

מסיר את הקובץ מה-staging אבל לא מוחק אותו מהתיקייה

Add + Commit

ניתן לקצר את התהליך ולבצע Add ו-Commit בפקודה אחת:

```
git commit -am "index.html"
```

מוסיף ושומר את כל הקבצים שהיו במעקב בקומיט אחד

צפייה בהיסטוריית קומיטים

```
git log
```

מציג את רשימת הקומיטים שבוצעו במאגר הנוכחי

כל קומיט כולל:



מזהה ייחודי (SHA)



מחבר



תאריך



הודעה

- ניתן להשתמש בכמה תווים ראשונים של ה-SHA בלבד כדי לזהות קומיט.

חזרה לקומיט מסוים

לפעמים נרצה לחזור אחורה בזמן ולראות מצב קודם של הפרויקט, או אפילו למחוק שינויים שנעשו.

צפייה בקומיט ישן

```
<git checkout <commit_id
```

מעביר אותך למצב של אותו קומיט (Detached HEAD)

איפוס מוחלט לקומיט

```
<git reset --hard <commit_id
```

מחזיר את הברנצ' למצב ההוא ומוחק שינויים מאוחרים יותר

Commit vs Commit --amend

git commit

```
git commit -m "Add feature"
```

יוצר קומיט חדש עם השינויים שנמצאים ב-staging

✔️ שימוש: לשינויים חדשים ונפרדים

git commit --amend

```
git commit --amend
```

עורך את הקומיט האחרון, מוסיף קבצים ששכחנו או משנה הודעה

✔️ שימוש: לתיקון טעויות, שמירה על היסטוריה נקייה, מקל על rebase

⚠️ אם הקומיט כבר נדחף ל-remote, יש צורך ב-force push:

```
git push -f origin <branch>
```

דוגמה מלאה

הנה תהליך שלם של שימוש ב-`commit --amend`:

1 קומיט ראשוני

```
"git commit -m "Add feature
```

2 הוספת קובץ ששכחנו

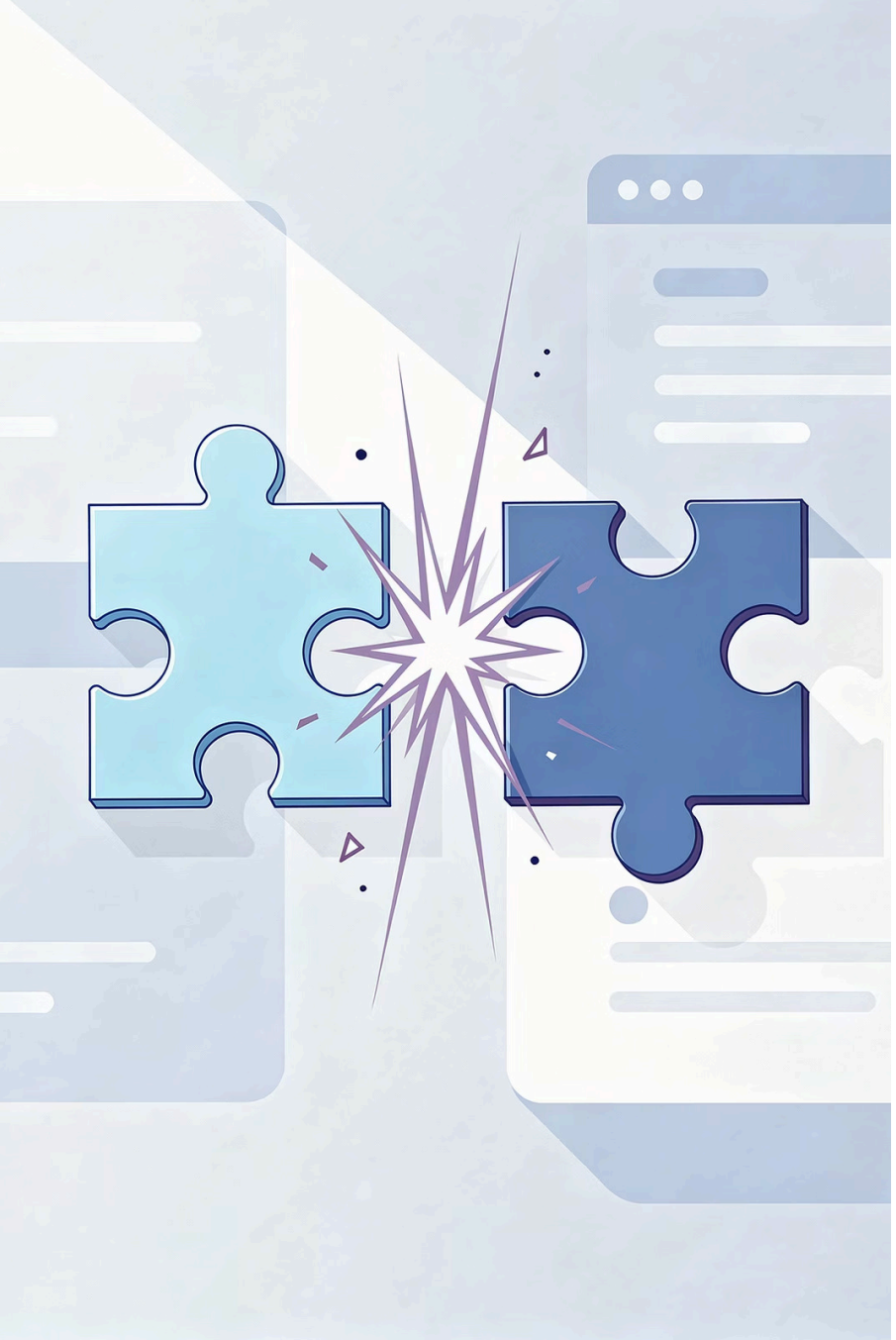
```
git add missed_file.js
```

3 עדכון הקומיט האחרון במקום ליצור חדש

```
git commit --amend
```

4 דחיפה בכפייה ל-remote

```
git push -f origin main
```



קונפליקטים, בסוף הכול מסתדר...

קונפליקטים בגיט הם מצב שבו שני משתמשים או יותר עשו שינויים באותו קובץ או באותו קטע של קוד. כאשר שני משתמשים עובדים על אותו קובץ, גיט עוקב אחר כל שינוי שנעשה בקובץ. כאשר אחד המשתמשים מנסה לגרום לשינויים שלו להתמזג עם השינויים של משתמש אחר, גיט מזהה קונפליקט אם שני המשתמשים עשו שינויים באותו קטע של קוד.

שני סוגי קונפליקטים

נבדיל בין שני סוגי קונפליקטים:

1

קונפליקט שנפתר בצורה אוטומטית

קונפליקט כזה קורה כאשר אתם שיניתם שורות בקובץ מסוים וחבריכם לפרויקט לא נגע באותו קובץ.

ההסתכלות של גיט – נוסיף את השורות שלכם לגרסה שנמצאת אצל החבר כאשר הוא יבצע משיכה. (מה שעשיתן בש"ב)

2

קונפליקט שאתם צריכים לפתור בין חברי הצוות

קונפליקט שקורה כשאתם וחבר צוות משנים את אותו הקובץ ואת אותן השורות. אז גיט נכנס לדילמה ושואל – איזה מהשינויים לקחת: שלכם? של החבר?

3 דרכים לפתור קונפליקטים

01

בחירת השינויים של משתמש אחד

הדרך הפשוטה ביותר לפתור קונפליקט היא לבחור את השינויים של משתמש אחד ולהתעלם משינויי המשתמש השני.

02

בחירת השינויים של שני המשתמשים

דרך נוספת לפתור קונפליקט היא לבחור את השינויים של שני המשתמשים. לדוגמה, אם משתמש אחד מוסיף שורה חדשה לקובץ, ומשתמש אחר מוחק את אותה שורה, ניתן לבחור לשמור את השורה החדשה של משתמש אחד, אך להוסיף הערה המציינת שהשורה נמחקה על ידי משתמש אחר.

03

שינוי השינויים של שני המשתמשים

דרך מורכבת יותר לפתור קונפליקט היא לשנות את השינויים של שני המשתמשים כך שהם יתמזגו בצורה חלקה. לדוגמה, אם משתמשים שני משתמשים עשו שינויים שונים באותה שורה של טקסט, ניתן לשנות את הטקסט כך ששני השינויים יתמזגו בצורה חלקה.

הדרך לפתור קונפליקט בגיט תלויה בנסיבות הספציפיות של הקונפליקט. אם הקונפליקט הוא פשוט, ניתן לפתור אותו על ידי בחירת השינויים של משתמש אחד או שני המשתמשים. אם הקונפליקט הוא מורכב יותר, ייתכן שיהיה צורך לשנות את השינויים של שני המשתמשים כדי שהם יתמזגו בצורה חלקה.

איך להימנע מבלאגן מיותר?

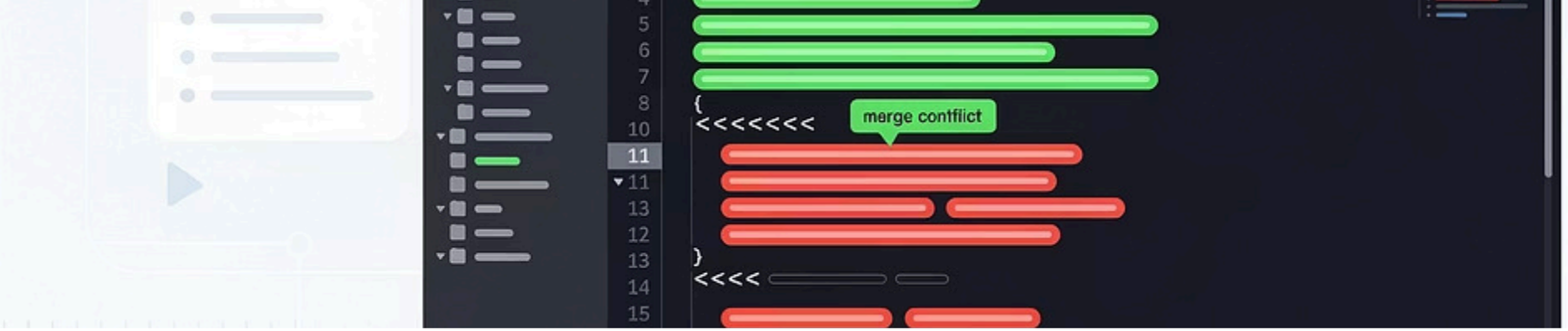
> הדרך הכי טובה להמנע מבלאגן מיותר היא ליצור קשר עם הצד השני ולשאול אם נכנס קומיט חדש.

הסיבה: קונפליקטים יכולים להיות מאוד לא סימפטיים (בייחוד כשיש הרבה מהם). לכן, עדיף לפתור אותם כשהם קטנים.

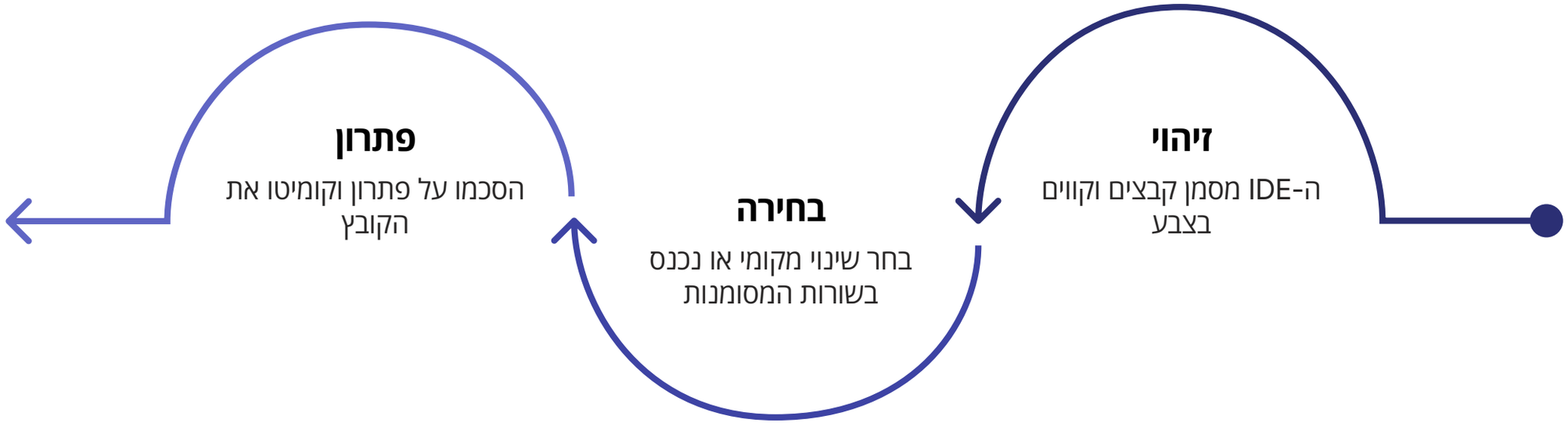
פתרון קונפליקטים – איך עושים את זה?

פתרון קונפליקטים נעשה מול סביבת העבודה שבה אנו עובדים. אם יש קונפליקט, הסביבה מזהה אותו ומסמנת בצבע את הקבצים שעברו שינוי.

כשלוחצים על אותם קבצים, השורות שעברו שינוי שהוביל לקונפליקט מוארות, ויש אפשרות לבחור בין השינוי הנוכחי לבין השינוי שזה עתה נכנס. בוחרים את השינוי בהתאם למה שהוסכם עם חבר הצוות וזהו זה, הקונפליקט נפתר!



פתרון קונפליקטים – סיכום התהליך



הסביבה מזהה את הקונפליקט, מסמנת את הקבצים, ומאפשרת לבחור בין השינויים – פשוט ומהיר!

ש.ב. ☹️

המשימה

כל זוג בנות, בת אחת פותחת פרויקט web api ומשתפת את השנייה.

שתיכן יחד עובדות על אותו הקובץ ופותרות קונפליקטים יחד!!

הנחיות:

- כל אחת עושה פונ' אחרת (ניתוב אחר, שם אחר)
- בנוסף שתיכן עושות פונ' getString ותבחרו את זאת שמחזירה string הכי מעניין